

Ingeniería de software

Evaluaciones	Porcentaje	Dia
Certamen 1	45%	
Certamen 2		
Test 1	10%	
Test 2		
Proyecto avance 1	45%	
Proyecto avance 2		



Condiciones del proyecto (tradicional-externo)

- 4 Integrantes
- Problemática o mejora
 - Datos de contacto del cliente y de los usuarios
 - Relacionada en el área de gestión y toma de decisiones
- Infraestructura para el cliente
- Identificar :
 - Requerimientos
 - Objetivos específicos
 - Objetivos generales
- Carta gantt

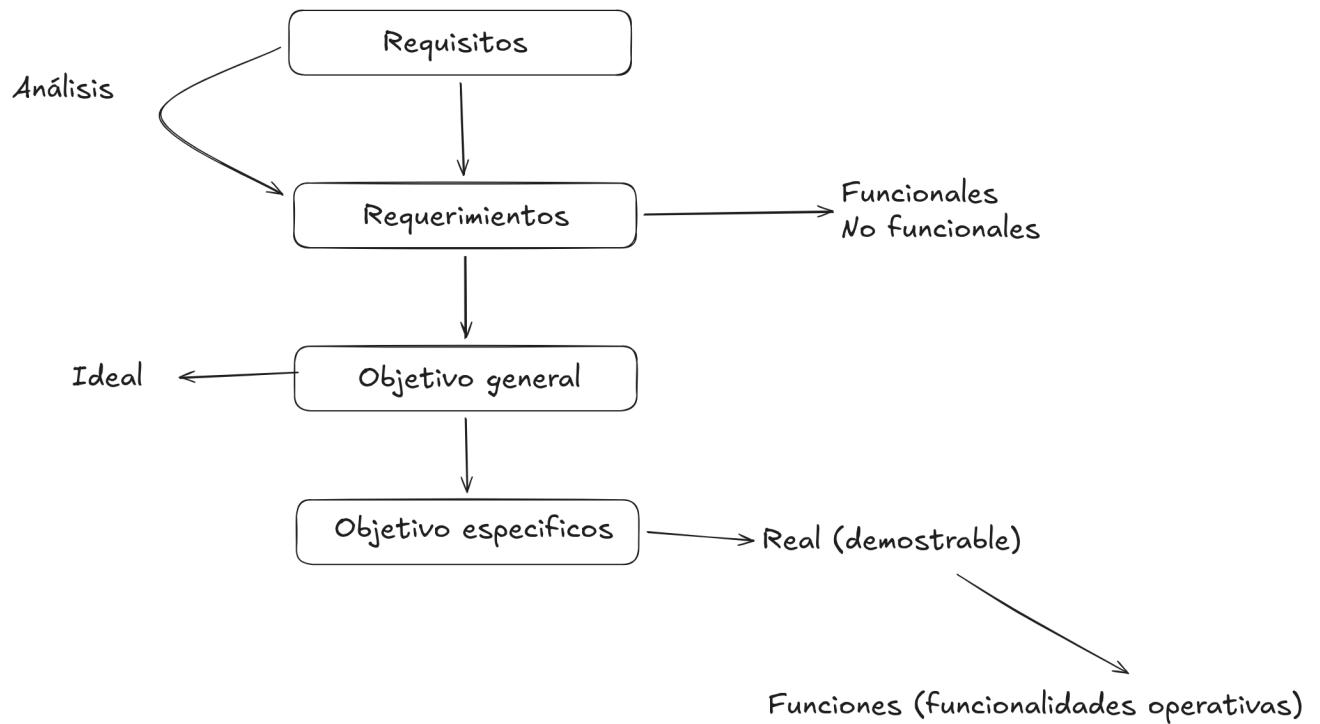
Condiciones del proyecto (tradicional-interno)

- 4 Integrantes
- Problemática o mejora
 - Datos de contacto del cliente y de los usuarios
 - Relacionada en el área de gestión y toma de decisiones
- Infraestructura para el cliente
- Identificar :
 - Requerimientos
 - Objetivos específicos
 - Objetivos generales
- Carta gantt
- Cliente interno
- Estimación de recursos

Condiciones del proyecto (innovación)

- 4 Integrantes
- Análisis de mercado
- Análisis técnico
- Roadmap

- Prototipo



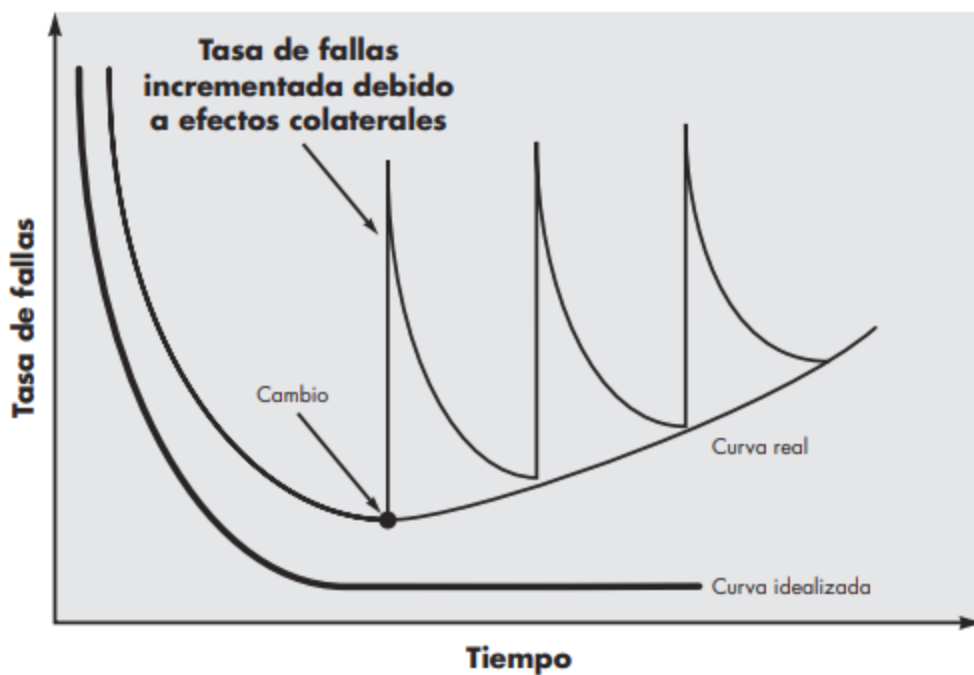
Recordar

- Tradicional
- Evolutivo
- Desarrollo rápido, etc
- Modelo o metodología de desarrollo
- Puede establecer estrategias que ven lineamientos de mas de un modelo
- Monolítico?
- traer hojas blancas
- <https://profesorezequielruizgarcia.wordpress.com/wp-content/uploads/2015/01/ingenieria-del-software-un-enfoque-practico-roger-s-pressman.pdf>

Capítulo 1

Falla

En el desarrollo de software tenemos distintas fases y también posibles problemas, estas son las curvas de desarrollo.

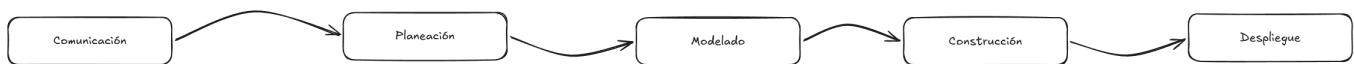


Las 7 grandes categorías

hay 7 grandes categorías de software actualmente, los cuales son:

1. Software de sistemas: Programas para dar servicios a otros programas, interactúan frecuentemente con el hardware.
 - Ejemplo: Sistemas operativos, Compiladores, etc.
2. Software de aplicación: Resuelven necesidades específicas de una empresa, datos comerciales, técnicos, etc.
 - Ejemplo: Software de transacción, control de procesos, etc.
3. Software de ingeniería y ciencias: Algoritmos con gran capacidad de procesar datos, habitualmente de tipo numéricos.
 - Ejemplo: Análisis de deformaciones, sistemas del transbordador espacial, etc.
4. Software incrustado: Piezas de software que residen en otros sistemas y sirven para controlar cosas específicas.
 - Ejemplo: Tablero de microondas, etc.
5. Software de línea de productos: Es creado para proporcionar de una capacidad específica a un público masivo y variado.
 - Ejemplo: Control de inventario, hojas de cálculo, etc.
6. Aplicaciones web: diversas por lo que es imposible especificar un propósito general de ellas, solo podemos decir que es un conjunto de páginas.
7. Software de inteligencia artificial: Uso de algoritmos no numéricos para resolver problemas complejos.

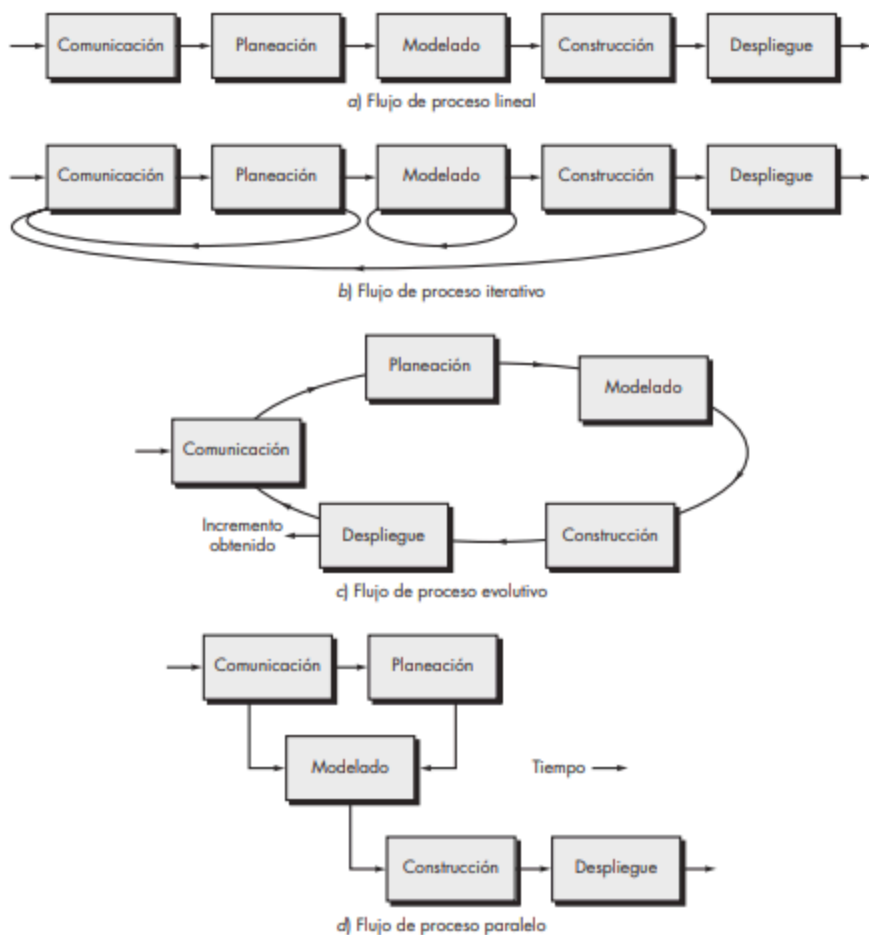
Proceso del software



- Comunicación: Es importante al empezar el proyecto tener una comunicación tanto con el equipo como con el cliente.
- Planeación: Es como un mapa a seguir a la hora de desarrollar un programa, describe las tareas técnicas, los riesgos, los recursos, los productos del trabajo y la programación de las actividades.
- Modelado: Es la creación de un modelo para entender los mejores requerimientos.
- Construcción: Es la generación de código y la corrección de errores.
- Despliegue: Se entrega el producto y se espera una retroalimentación.

Capítulo 2

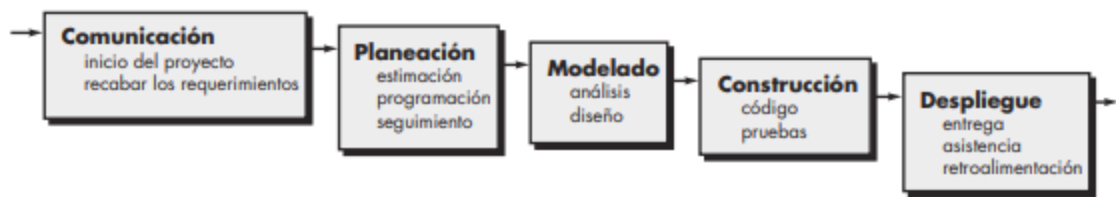
Flujos de procesos



Modelos de proceso prescriptivo

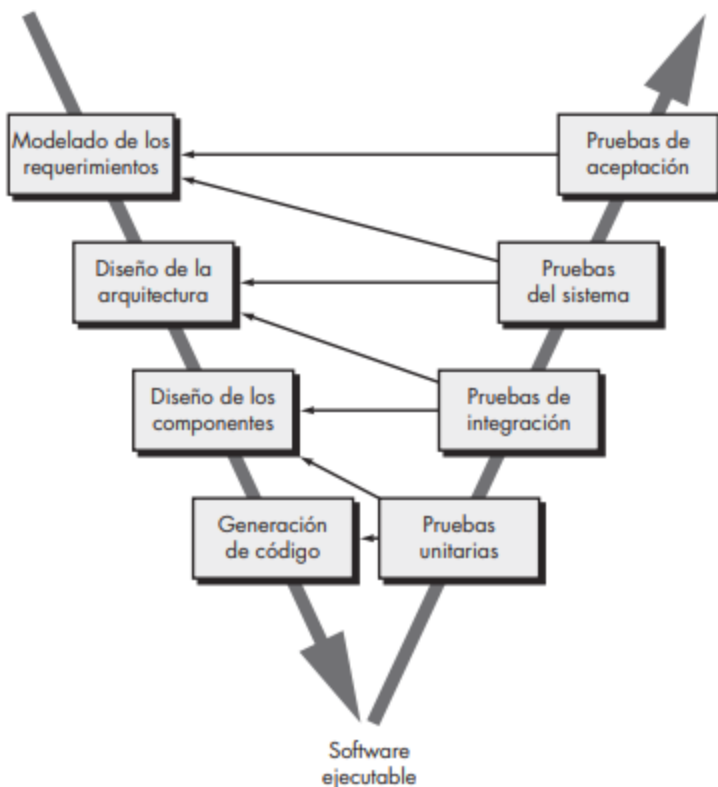
Busca generar orden y estructura, uno de los primeros modelos

Modelo de la cascada



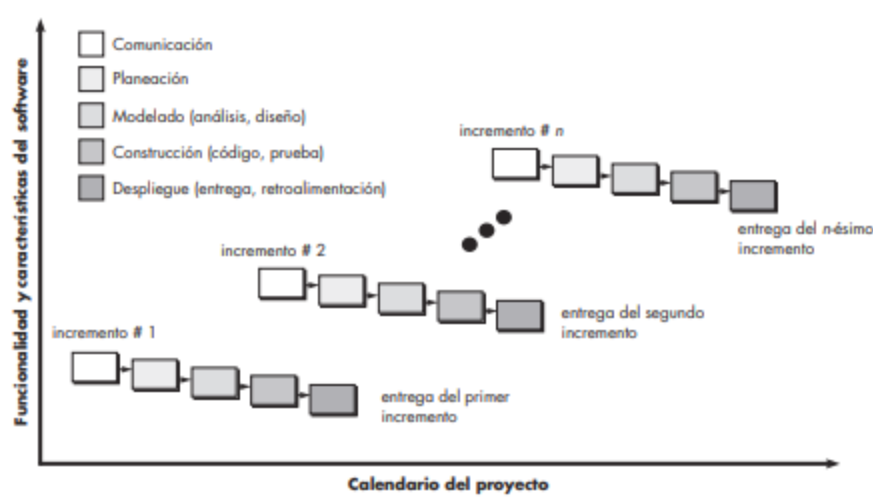
Es un enfoque sistemático y secuencial, es una especie de proceso "ideal" puesto que se asume que la comunicación hasta el despliegue fueron perfectos a la primera. Pero en la vida real estos es muy raro y casi improbable, por eso se generó la siguiente variante:

Modelo en V



Este modelo surge como una manera de verificar la calidad del producto, se hacen pruebas para garantizar un proyecto de calidad

Modelos de procesos incremental

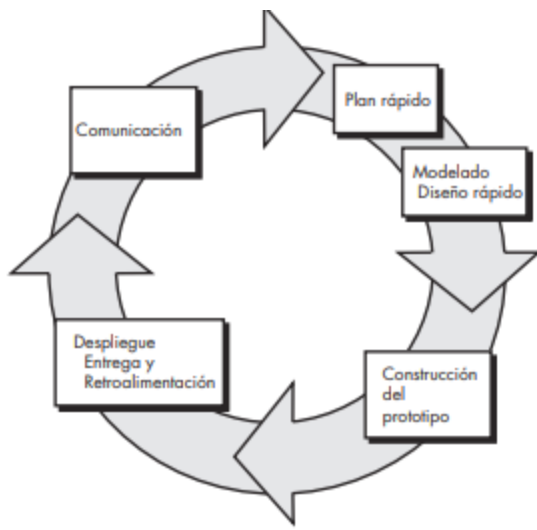


En este modelo se trata de desarrollar "entregas" donde se separan en incrementos o entregas, por decir así se realiza el proceso básico, pero enfocado en la primera entrega a lo fundamental, en la siguiente entrega se realizan más cambios y se agregan más cosas basándose en la retroalimentación de la primera entrega.

Modelos de procesos evolutivos

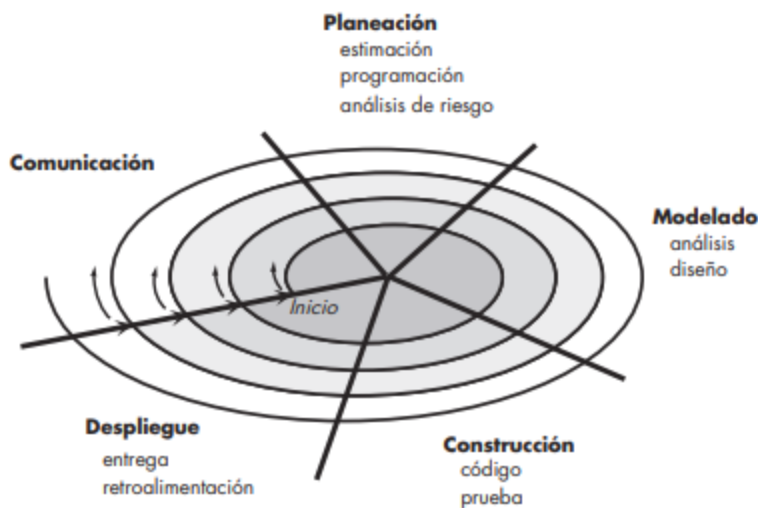
Los modelos evolutivos son iterativos y se caracterizan por la manera que les permite desarrollar cada vez versiones más completas del software, en donde encontramos:

Hacer prototipos



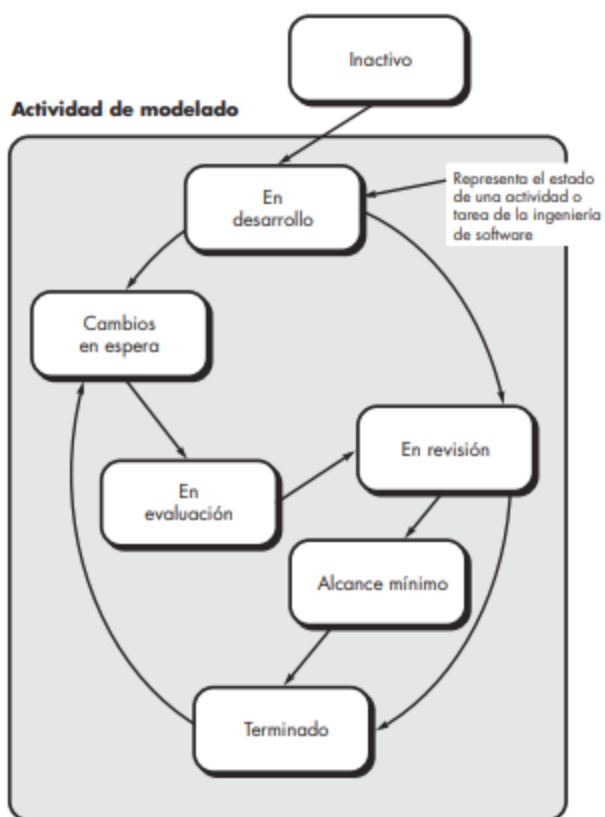
El prototipo es un mecanismo que tenemos para identificar requerimientos, un objeto desechable, puesto que sirve para realizar demostraciones, es un producto con fallas, cosas a mejorar o a optimizar, de estos prototipos surgirá el producto definitivo, pero un prototipo idealmente nunca será el producto final.

El modelo espiral



Este modelo tiene el potencial de lograr un desarrollo rápido con versiones cada vez más completas, por cada revolución el riesgo aumenta la primera vuelta es para desarrollar las especificaciones del producto, luego prototipos y luego software cada vez más sofisticado, con cada iteración son más requerimientos, puesto que la espiral es más grande, este se considera un sistema realista, puesto que permite desarrollar modelos, es sistemático y también puede lograr que un proyecto no finalice en la entrega final, sino que se siga desarrollando, no es perfecto, puesto que se necesita mucha experiencia para lograrlo.

Modelos concurrentes



Es como una red más que un modelo lineal, los estados en comunicación, modelado, etc. tienen a su vez sub estados que podrían ser inactivo, en revisión, etc. Entonces podemos decir que todo está ocurriendo al mismo tiempo solo que están en distintos estados, y un cambio en un estado desencadena cambios en otros, esto es útil debido a que los clientes muchas veces cambian a mitad del proceso de requerimientos, o aparecen errores, etc.

—  —

Modelos de proceso especializado

Similares a los procesos anteriormente enseñados solo que este se enfoca en proceso más especializado o definido muy específicamente.

Desarrollo basado en componentes

También llamado COTS por sus siglas en inglés, es similar al modelo espiral, ya que es evolutivo e iterativo, pero la diferencia es que en este modelo se construyen aplicaciones con base en fragmentos de software, esto se puede lograr con piezas de software convencional o clases orientadas a objetos o paquetes de clases, esto ofrece una ventaja, pues se desarrolla producto más rápido y con menor costo.

El modelo de métodos formales

Este sistema funciona como matemáticas, es estricto y no deja a interpretación nada y se basa en 3 claves:

- Especificar: Define el sistema como matemáticas.
 - Desarrollar: Construye el software basado en esas especificaciones.
 - Verificar: Comprueba que el código cumple con exactamente lo definido.
- Las desventajas de este sistema es que lento, caro, no mucha gente sabe usarlo y es difícil

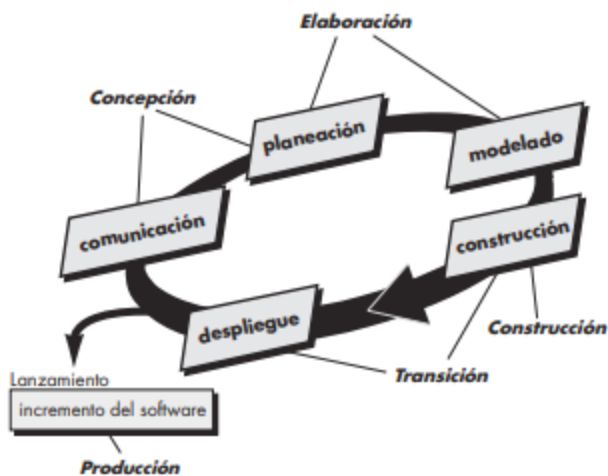
de explicar al cliente.

Desarrollo de software orientado a aspectos

Cuando uno programa habitualmente se basa en programación orientada a objetos, pero donde quedan las preocupaciones generales, por ejemplo la seguridad, para esto nace este modelo orientado a aspectos, es como una capa más arriba que el POO, puesto que este agrupa preocupaciones globales, los define y los aplica, por decir encapsulamos problemas, este modelo atraviesa todo el sistema y afecta múltiples partes a la vez.

3

El proceso unificado



Este proceso trata de ser una version de todo lo mejor de los modelos antes vistos, por lo que tiene que ser:

- Iterativo
- Incremental
- Basado en casos de uso
- Centrado en arquitectura

Creamos casos de usos y creamos una estructura para el sistema de esta manera tratamos de organizar todo de la mejor manera y también vamos iterando para crear mejores versiones, hay 5 fases:

1. Concepción

- Hablar con el cliente
- Entender el problema
- Definir los casos de usos iniciales
- Bosquejar la arquitectura

2. Elaboración

- Refinamos los casos de uso
- Definimos la arquitectura de manera seria
- Analizamos riesgos

3. Construcción

- Implementamos funcionalidades
- Hacemos pruebas unitarias

- Integramos componentes

4. Transición

- Pruebas beta
- Feedback
- Correcciones
- Documentación

5. Producción

- Soporte
- Arreglar bugs
- Mejoras

Como dije antes es un modelo basado en iteraciones, por lo que en cada iteración se agregan más funciones o mejoras.



Modelos del procesos personal y del equipo

Modelos para estar cerca del equipo y de las personas que participan en el proyecto, no trata de hacerse a nivel corporativo ni organizacional.

Proceso personal del software (PPS)

En este sistema se busca optimizar a las personas por individual, cada uno tiene que hacer lo siguiente:

- Planificar tu trabajo
- Mides cuanto tardas
- Registrar errores
- Analizas tu rendimiento

En resumen busca tú optimización como si tú fueras el sistema y funciona en las siguientes fases:

1. Planeación

- Estimas cuantos vas a tardar.
- Cuanto código vas a programar.
- Cuantos errores crees que tendrás.

2. Diseño de alto nivel

- Piensas en la solución.
- Haces estructuras.
- Y si no estas seguro haces prototipos.

3. Revisión del diseño

- Revisas tu diseño antes de programar.
- Buscas errores conceptuales.

4. Desarrollo

- Programas
- Pruebas

- Registras cuanto tardas
- Anotas errores

5. Post mortem

- Analizas todo.
- Cuanto te equivocaste en las estimaciones.
- Qué errores cometiste.
- Y por qué ocurrieron.

No se suele usar mucho puesto que toma mucho tiempo, se tiene que medir todo y requiere mucha disciplina.

Proceso del Equipo de Software (PES)

Similar al anterior, pero enfocado en el equipo completo, se busca lograr un equipo autodirigido el cual cuenta con las siguientes características:

- Define sus metas
- Organiza su trabajo
- Mide su rendimiento
- Mejora su proceso
para eso el equipo debe:
- Planificar el proyecto
- Definir roles
- Medir la productividad
- Controlar la calidad
- Gestionar riesgos

Sus fases son similares al PPS, pero aplicadas a un equipo:

1. Inicio del proyecto

- Objetivos
- Planificación
- Roles

2. Diseño

- Arquitectura
- Decisiones técnicas

3. Implementación

- Desarrollo de código

4. Integración y pruebas

- Unir todo
- Testear

5. Post mortem

- Analizar resultados
- Mejorar procesos

Tiene las mismas dificultades que el modelo anterior.

Conceptos

Dado todo lo anteriormente aprendido surgen los siguientes dos conceptos:

Tecnología del proceso: En resumen nos dice que necesitamos usar herramientas que nos ayuden a organizarnos , de esta manera nos permite ser mas flexibles y poder usar el modelo que mas se adapte a nuestras necesidades, tenemos un mayor control y podemos visualizar avances, tenemos por ejemplo las siguientes herramientas:

- Gestión de proyecto(Trello, Jira)
- Modelado(Diagramas, UML)
- Seguimiento(Metricas y tiempo)
- Calidad(Testing y control de errores)

Producto vs proceso: Estos dos conceptos no son opuestos, se complementan, se necesitan para desarrollar un producto de calidad.